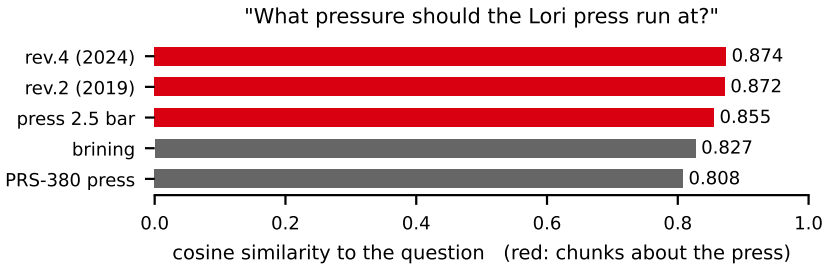
Retrieval-Augmented Generation

Part 3: answering, and knowing whether you answered well

Your retrieval works now

Two lectures of work later, the engineer asks again:



The correct, current procedure - press manual revision 4 - comes back at **rank 1**. Measured on our 20-chunk corpus with a real embedding model.

And here is the answer it produced

> *"The Lori press runs at 2.5 bar. The pressure stays at 2.5 bar throughout the pressing stage. This is the setting used for Lori cheese, and the same setting applies to every press in the factory."*

Every sentence is supported by a chunk that was actually retrieved. And the answer is wrong: the **2024** revision raises the pressure to **3.2 bar** after twenty minutes. The model quoted the **2019** revision, which was also sitting in the context.

The answer text is an illustration of this failure pattern, not a captured model run. The retrieval above it is measured.

Every retrieval number was perfect

What we measured	Score	
Did the right chunk come back first?	1.00	**perfect**
How high did it rank, on average?	1.00	**perfect**
Was the whole ordering sensible?	0.97	**near perfect**
Was the answer right?	**no**	

You can score a retrieval system perfectly and still ship a system that gives wrong answers. Retrieval metrics measure **what arrived in the prompt**. They say nothing about **what the model did with it**, and nothing about whether the corpus itself is consistent.

By the end of this lecture you will know what those three numbers are, and why all three missed.

Outline

Assembling the context

Prompting so the answer stays grounded

How this breaks

Measuring retrieval

Measuring generation

Building an eval set from nothing

Beyond the naive pipeline

When not to use RAG

Assembling the context

You have a ranked list. The model needs a prompt. That is not the same object.

Three decisions between the list and the prompt

The retriever hands you 100 chunks with scores. You cannot paste 100 chunks. Someone has to decide:

How many?

A token budget you are paying for on every question.

In what order?

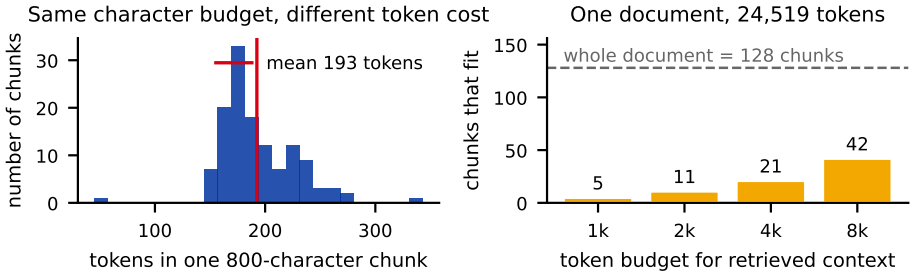
Position inside the prompt is not neutral.

Which are the same?

Overlapping chunks and old revisions repeat the same sentence.

This step has no model in it and no maths. It is plumbing. It is also where a surprising share of real RAG bugs live, because nobody thinks it is worth attention.

How much actually fits



Real token counts from one technical document of 24,519 tokens, cut into 800-character chunks. Equal-sized chunks are *not* equal-cost: 150 to 340 tokens each, because tables and formulas tokenize badly.

A 2,000-token context holds **11 of the 128 chunks** - 8.6% of one document. Your budget is far smaller than it feels.

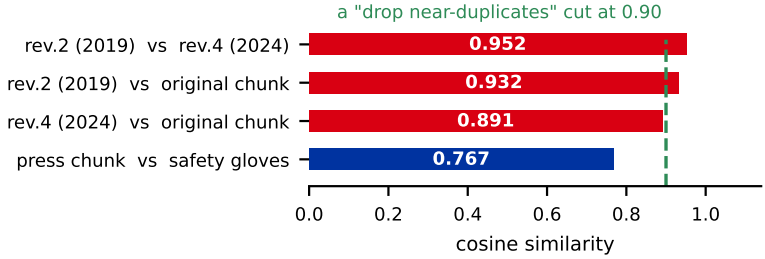
Where in the prompt matters

Models do not read a prompt uniformly. A widely reproduced result (Liu et al., 2023) is that accuracy is highest when the needed passage sits near the **start** or the **end** of a long context, and lowest when it is buried in the middle.

Do	**Why**
Best chunk first, next best last	the ends are where the model reads best
Keep the context short	fewer places for the answer to hide
Label each chunk with an id	so the model can cite it
Include the metadata line	date and source change the reading

Do not sort your chunks alphabetically, by document name, or by upload date because it was easier to implement. Sort by score, and know that you did.

The same sentence, three times



Measured on our corpus. The three chunks about press pressure - two revisions and the original paragraph - all sit above 0.89. Two chunks with nothing in common still sit at **0.767**.

The dedup trap

The obvious rule is: drop any chunk that is more than 0.90 similar to one you already kept. Look at what that rule does here.

What it deletes

Revision 2 and revision 4 score 0.952 against each other. One of them gets dropped. Nothing in a cosine tells you which one is current.

Why the threshold is a guess

Unrelated text already scores 0.767 on this model. The usable range is 0.77 to 1.00, not 0 to 1, and it moves when you change model.

Deduplicate on **metadata first** - same document, same section, newer revision wins - and only then on similarity. A date is a fact. A cosine is an opinion.

Prompting so the answer stays grounded

The model will happily answer from memory. You have to make that the harder option.

What all of that actually produces

Three decisions, one filter, three chunks. This is the literal string the model receives - nothing is hidden behind an abstraction:

```
Answer the question using ONLY the context below.
Cite the block number you used, like [1].
If the context does not contain the answer, reply exactly:  I don't know.

CONTEXT
[1] (press_manual, rev 4, 2024)
Press manual revision 4, issued 2024:  the Lori press starts at 2.5 bar and is
raised to 3.2 bar after twenty minutes.

[2] (equipment_log, rev 1, 2022)
Model PRS-400 replaces the older PRS-220 press on line two.

QUESTION
What pressure should the Lori press run at?

ANSWER
```

Produced by `ml/ch17_rag/rag_demo.py`, which you can run and change. Retrieval, filtering and this prompt are about 140 lines in total.

The prompt is a contract

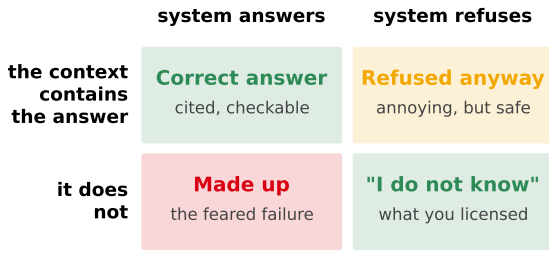
```
Answer using ONLY the passages below.
After every sentence, cite the passage id, like [3].
If the passages do not contain the answer, reply exactly:
"I do not know from the documents I have."

[1] (rev.4, 2024) The Lori press starts at 2.5 bar, raised to
    3.2 bar after twenty minutes.
[2] (rev.2, 2019) The Lori press is held at 2.5 bar throughout.

Question:  What pressure should the Lori press run at?
```

Three separate instructions: **restrict the source**, **demand a citation**, **license a refusal**. Drop any one of them and you lose a different guarantee.

Licensing “I do not know”



Tighten the prompt and "Made up" shrinks - but "Refused anyway" grows.

A model that never refuses will invent. A model that refuses too easily is useless. This is a **trade-off you choose**, and where you set it depends on what a wrong answer costs: a wrong pressure setting breaks a press, a wrong film recommendation does not.

Making the refusal reachable

"Say you do not know if you are unsure" does not work well, because the model has no calibrated sense of unsure. Give it a mechanical test instead:

- **Name the exact refusal string.** A fixed sentence you can detect in code and count. "I do not know" phrased twelve ways is not measurable.
- **Make refusal a valid answer, not a failure.** If the prompt only shows examples of successful answers, refusing looks off-distribution.
- **Refuse per claim, not per question.** "The pressure is 2.5 bar [1]; the humidity is not in these documents" beats refusing the whole thing.
- **Add a retrieval-score floor.** If the best chunk scores far below what a real match scores, do not call the model at all.

The score floor needs a number, and that number is corpus-specific. Ours would have to sit above 0.767, because that is what two unrelated chunks already score.

What a citation does and does not prove

A citation gives you

A place to look.

A reader who can check in ten seconds.

A trail when the answer is later found to be wrong.

A citation does not give you

Any guarantee that the cited passage says it.

Any guarantee that the passage is current.

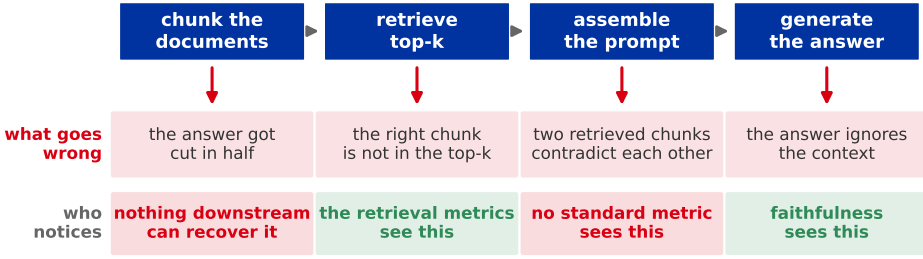
Any guarantee the model used it rather than attached it afterwards.

Citations are a **verification aid**, not a correctness mechanism. The system that produces the citation is the same system you do not trust. That is the whole problem with checking a model's work using the model.

How this breaks

Four places a pipeline leaks, and the uncomfortable fact that your metrics only see two of them.

Where a pipeline leaks



The green boxes are the ones a standard evaluation catches. The two hardest failures - a destroyed chunk and a self-contradicting corpus - are in red.

The first two leaks you have already met: **chunking** was lecture 41's recall ceiling, **retrieval** was most of lecture 42. The next three slides cover what is left - the failures that appear *after* retrieval succeeded.

Failure 1: retrieved, and then ignored

The correct chunk is in the prompt. The model answers from its training data anyway.

> **Context:** "The Lori press starts at 2.5 bar. . ."
> **Answer:** "Cheese presses typically operate between 4 and 6 bar."

It happens most when the retrieved fact **contradicts what the model already believes**, and when the context is long enough that the relevant line is easy to skip.

Cheap defences: a shorter context, the relevant chunk placed first, and an explicit "only from the passages" instruction. All three make the training-data answer the harder path.

Failure 2: right answer, wrong citation

The answer is correct. The bracket after it points at the wrong passage.

> *"The ripening cellars are held at 10 degrees [2]."*
>
> . . . where passage [2] is the brine tank, and passage [4] is the cellar.

This looks harmless and is not. It is the failure that **destroys trust fastest**, because it is the one a careful reader finds first: they click the citation, it does not say that, and now they doubt everything else on the page.

It is also easy to check mechanically: for every cited id, test whether that passage actually supports that sentence. You do not need a human for this, and almost nobody does it.

Failure 3: the sources disagree

This is the one from the cold open, and it is different in kind from the other two.

> **[1] rev.4, 2024:** starts at 2.5 bar, raised to 3.2 bar after twenty minutes.
> **[2] rev.2, 2019:** held at 2.5 bar for the whole pressing stage.

Retrieval did its job: both relevant chunks came back. Generation did its job: the answer is grounded in a retrieved passage, with a citation. **The corpus is the thing that is broken.**

No metric in this lecture catches this, because every metric asks "is the answer supported by the context?" and the answer is **yes**. Only **metadata** fixes it: index the revision date, filter to current, and prefer the newest at retrieval time.

The cure, actually applied

We diagnosed the opening failure and prescribed a fix. Here it is running, on the same question, with one line changed:

Before - no filter

```
[1] rev 4, 2024
[2] rev 2, 2019
[3] press 2.5 bar
```

The superseded revision is **second**, 0.002 behind. The model quotes it, cites it correctly, and is wrong.

After - `current == True`

```
[1] rev 4, 2024
[2] press 2.5 bar
[3] brining
```

The 2019 revision never reaches the prompt. The answer is now the current procedure.

Nothing about the model, the embedder or the metrics changed. **One metadata field and one filter** fixed a failure that every retrieval number called perfect.

Measuring retrieval

Three numbers, each answering a different question. All three are computed by hand.

What you need before any number exists

Every metric here needs the same thing: a list of questions where somebody has written down **which chunk is the right one**.

Question	**Correct chunk**
"How small do we chop it up?"	curd cutting
"How is the raw milk made safe…?"	pasteurisation
"How hard does the machine squeeze the cheese?"	press, 2.5 bar

Run the retriever on each question, find the **position** of the correct chunk in the results, and every metric below is a function of those positions.

Where this list comes from when you have no users yet is a whole section of this lecture. For now, assume somebody wrote it.

recall@k: did it get into the prompt at all?

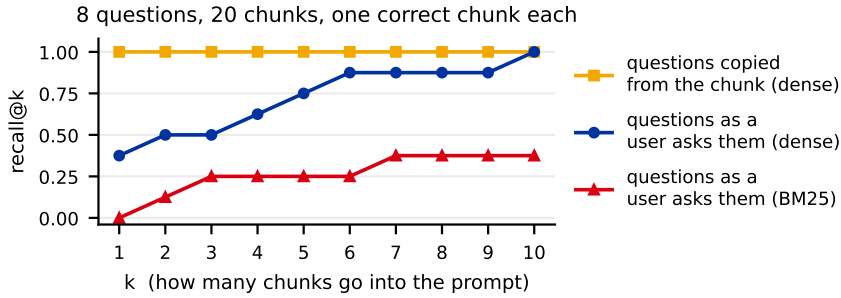
$$\text{recall@}k \;=\; \frac{\text{questions whose correct chunk is in the top } k}{\text{all questions}}$$

By hand, on the three questions above, with the positions our retriever actually produced:

Question	Position	in top 1?	in top 5?
chop it up	1	yes	yes
raw milk made safe	2	no	yes
squeeze the cheese	10	no	no
		1/3 = 0.33	**2/3 = 0.67**

This is **the** metric for RAG. The generator reads all k chunks, so it does not care whether the answer was 1st or 5th - only whether it was there.

recall@k over the whole set

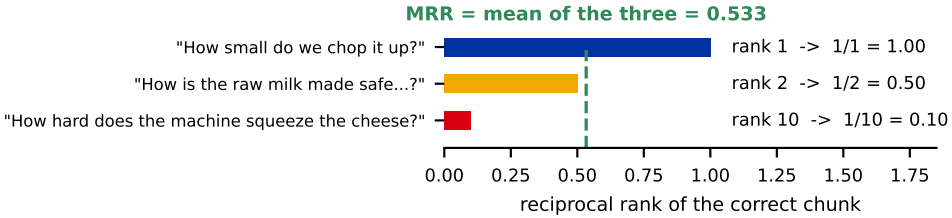


Measured. Ignore the flat orange line for now - it comes back in two sections.

recall@k only ever goes up with k. That is not a free win: bigger k means more tokens, more cost, and more places for the answer to hide.

Mean Reciprocal Rank: how high did it land?

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q}$$



Mean Reciprocal Rank (MRR) punishes depth harshly: rank 1 is worth ten times rank 10, and rank 10 and rank 50 are almost the same number. It only looks at the *first* correct chunk.

When one hit is not the whole story

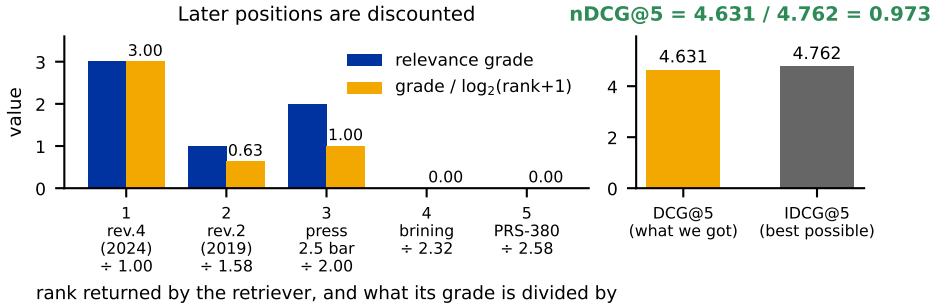
recall@k and MRR both treat relevance as **yes or no**. Real questions are not like that:

Chunk	How useful
Press manual rev.4, 2024 - the current, complete procedure	**3**
"The Lori press operates at 2.5 bar…" - part of the answer	**2**
Press manual rev.2, 2019 - right topic, superseded	**1**
Brining, PRS-380 - unrelated	**0**

These grades are a **judgement call**, not a formula. Someone else could defend giving the superseded revision a 0. Everything below is exact arithmetic on a subjective input, which is the honest description of *every* graded relevance metric.

We want a metric that (a) uses these **grades**, and (b) rewards putting the 3 above the 1. That is **normalized Discounted Cumulative Gain (nDCG)**.

nDCG, computed



Discounted Cumulative Gain (DCG) adds up the grades, each divided by $\log_2(\text{rank}+1)$. **Ideal DCG (IDCG)** is the same sum for the best possible ordering, here $3, 2, 1, 0, 0$. Dividing one by the other puts the score on $[0, 1]$ so queries with different numbers of relevant chunks are comparable.

Reading nDCG honestly

Our retriever put a superseded 2019 document above a chunk that was more useful, and still scored **0.97**.

$$\text{nDCG@5} = \frac{4.631}{4.762} = 0.973$$

Two things are true at once:

- Given the grades, 0.97 is **correct**. Swapping ranks 2 and 3 is a small mistake and the metric says so.
- The consequence of that small mistake was a **wrong answer**, because the generator read the 2019 sentence and believed it.

A retrieval metric scores the *ordering*. It has no idea what the text inside the chunks says, so it cannot know that rank 2 and rank 1 disagree with each other.

Which one to actually report

Metric	Answers	Use it when
recall@k	Did the answer make it into the prompt?	Always. This is the one that caps your whole system.
MRR	How near the top was the first good chunk?	You show a ranked list to a person, or k is very small.
nDCG	Was the whole ordering sensible, with grades?	You have graded relevance and a reranker to compare.

Report **recall@k at the k you actually use in production**. A recall@100 of 0.99 is meaningless if you paste 5 chunks into the prompt.

Measuring generation

Now the harder half: scoring an answer when nobody wrote down what the right answer was.

Three questions retrieval cannot answer

Faithfulness

Is the answer supported by the context we gave it?

Answer relevance

Does the answer address the question that was asked?

Context precision

originally called

context relevance

Was the context worth the tokens it cost?

These three come from **Ragas** (Retrieval Augmented Generation Assessment), 2023. Their point is that all three can be computed **without a reference answer** - which matters, because you do not have one.

Faithfulness: break the answer into claims

"The Lori press runs at 2.5 bar."	**supported by the context**
"The pressure stays at 2.5 bar throughout the pressing stage."	**supported by the context**
"This is the setting used for Lori cheese."	**supported by the context**
"The same setting applies to every press in the factory."	**NOT in the context**

faithfulness = 3 supported / 4 statements = 0.75

$$F = \frac{|V|}{|S|} = \text{supported statements} \,/\, \text{all statements}$$

Split the answer into standalone statements, then check each one against the context on its own.

What faithfulness measured, and what it missed

It caught the over-generalisation: nothing in the context says the setting applies to every press. Good.

It marked *"the pressure stays at 2.5 bar throughout the pressing stage"* as **supported** - and it is, by the 2019 revision that was sitting in the context.

Faithfulness checks each claim against the context. It never checks the context against itself. A perfectly faithful answer to an inconsistent corpus is still a wrong answer.

This is not a flaw in the metric. It is the metric doing exactly what it says, and you assuming it said more.

Answer relevance: run the question backwards

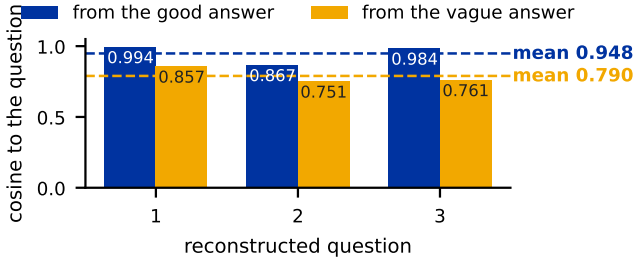
You have no reference answer, so you cannot compare to one. The trick:

1. Hide the question. Show a model only the **answer**, and ask it to write the question this answers.
2. Do that a few times, getting several reconstructed questions.
3. Measure how close those land to the **real** question, with the embedding cosine from the retrieval lecture.

If the answer really addressed the question, you can recover the question from it. A vague or padded answer could have been written for many questions, so the reconstructions drift.

Note what this does *not* check: nothing about truth. An answer can be confidently wrong and perfectly relevant.

Answer relevance, measured



Real cosines. A precise answer scores **0.948**; "pressing is important and the settings depend on the equipment" scores **0.790**.

The vague answer still scores 0.790, and two unrelated chunks scored 0.767 earlier. **The scale does not start at zero.** Only the comparison between two systems on the same data means anything.

Was the context worth its tokens?

The third number in its original form, **context relevance**: of the sentences you pasted, how many were needed?

Sentence in the cold-open context	Needed?
rev.4: starts at 2.5 bar, raised to 3.2 bar	yes
rev.2: held at 2.5 bar throughout	no
"The Lori press operates at 2.5 bar…"	yes
"Cheese brining follows the pressing stage."	no
"The brine tank is kept at 12 degrees…"	no
"The PRS-380 is rated for harder cheeses…"	no

$$\text{context relevance} = \frac{2}{6} = 0.33$$

Two thirds of what you paid for did nothing.

Context precision: reward putting them first

Its current form, **context precision**, works on **chunks** and cares about **order**:

$$\text{context precision@}K \;=\; \frac{\sum_{k=1}^{K} \text{precision@}k \cdot v_k}{\text{number of relevant chunks in the top } K}$$

where $v_k = 1$ if the chunk at position k is relevant, and precision@k is the fraction of the first k chunks that are relevant.

Two retrievals, same two relevant chunks out of five:

Relevant at	**Terms**	**Score**
ranks 1, 2	$\frac{1}{1} = 1.00$, $\frac{2}{2} = 1.00$	$(1.00 + 1.00)/2 = \mathbf{1.00}$
ranks 2, 4	$\frac{1}{2} = 0.50$, $\frac{2}{4} = 0.50$	$(0.50 + 0.50)/2 = \mathbf{0.50}$

Same chunks retrieved, half the score, because the junk was in front. This is the metric that tells you a **reranker** would help.

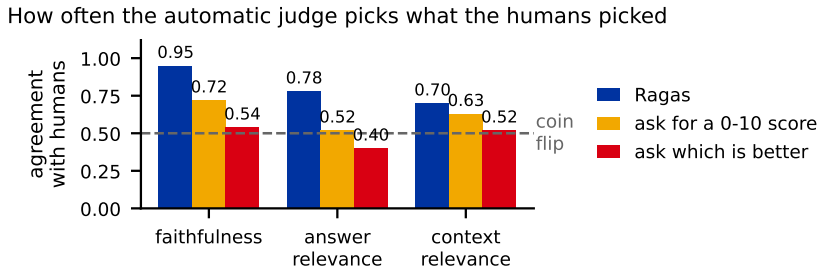
Every one of these needs a judge

Look at what the three metrics actually required:

Step	Who does it
Split an answer into standalone statements	a model
Decide whether a statement follows from the context	a model
Write the question that an answer answers	a model
Decide which sentences were needed	a model

This is **LLM-as-judge** (LLM-11), and it is not a shortcut anyone took out of laziness. There is no formula that decides whether a sentence follows from a paragraph. A human can do it; a human cannot do it 10,000 times a night, on every commit.

And every one of these judges is suspect



Reported by the Ragas authors on 50 Wikipedia questions, against two human annotators. The purpose-built pipeline agrees 0.95 of the time on faithfulness. Simply asking a model "which answer is better" agrees **0.40** of the time - worse than flipping a coin.

Using these judges

LLM-11 gave the general rules for judges: decompose the task, calibrate against people, compare instead of trusting absolute numbers, freeze the judge, and watch for position, length and self-preference biases. For RAG they mean:

- Ragas already **decomposes**: every metric above is built from yes/no checks on single statements - the first rule, built in.
- **Calibrate** on 50 of your own questions before trusting a faithfulness score on your documents.
- Report a change as "0.81 versus 0.74 last week, same judge, same questions", never as a bare "0.81".

Never let the judge be the same model, with the same prompt, as the generator. You are asking it to mark its own homework.

Which generation number to reach for

Six numbers is too many to compute every time. Reach for them in this order:

Symptom	Measure first	Because
Plausible but wrong	**recall@k**	if the chunk never arrived, nothing after it matters
Right chunk there, answer still wrong	**faithfulness**	separates "ignored the context" from "context was bad"
Faithful, still wrong	**look at the corpus**	no metric catches a corpus that contradicts itself
Answer drifts off the question	**answer relevance**	catches waffle that is technically supported
Costs too much, quality fine	**context precision**	how much of the prompt earned its tokens

Always upstream first. **recall@k is the cheapest and most upstream**, so it is almost always the right first measurement - and the one most often skipped in favour of staring at the generated text.

The cold open, scored

Metric	Score	What it was looking at
recall@1	1.00	the right chunk came back first
MRR	1.00	it came back first
nDCG@5	0.97	the ordering was nearly ideal
faithfulness	0.75	3 of 4 claims supported by the context
answer relevance	high	it is clearly an answer about press pressure
context relevance	0.33	most of the context was wasted
correct?	**no**	

The only number that looked bad was context relevance, and it was bad for an unrelated reason. **Metrics are necessary and not sufficient.** Read some of your system's answers, by hand, every week. There is no substitute and there never has been.

Building an eval set from nothing

You have a corpus, no users, and no labelled questions. Start anyway.

The obvious idea, and it is a good one

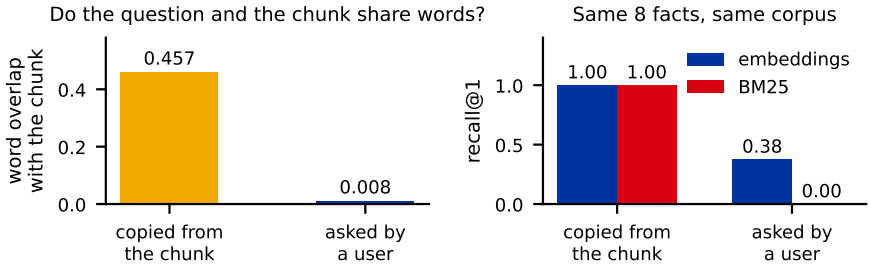
Take a chunk. Ask a model to write a question that this chunk answers. Now you have a labelled pair, because **you know which chunk it came from**.

> **Chunk:** "Ripening cellars are held at 10 degrees and 85 percent humidity."
> → **Question:** "At what temperature and humidity are ripening cellars held?"
> → **Correct chunk:** this one, by construction.

Two hundred chunks, two hundred questions, one afternoon, no annotators.

This is genuinely the right first move. It gets you from "no idea" to "measurable" in a day. But it has one specific defect, and it is worth a slide.

The defect, measured



Same eight facts, same corpus, same retriever. The generated questions reuse the chunk's own words, so both retrievers find them **every time**. Phrase the same eight questions the way a person would, and BM25 drops to **0.00** and embeddings to **0.38**.

Your eval set was measuring the wrong thing

A question copied from a chunk shares that chunk's vocabulary. Retrieval by shared words is then trivial. **This is nearly a tautology** - which is exactly why it is dangerous. It looks like a measurement.

Your dashboard reports recall@1 = 1.00. Your users experience something much worse. Nothing in the pipeline is broken and nothing in the dashboard is lying.

Honest caveat: our eight "as asked" questions were deliberately written to avoid the chunk's words, so 0.008 word overlap is an extreme. Real user questions land in between. The gap is smaller than this - it is never zero.

Fixing it

- **Instruct the generator to paraphrase.** "Ask as a factory worker who has never read this document would ask, using none of its technical terms."
- **Generate several styles per chunk.** Terse, rambling, misspelled, in Armenian, with the part number, without it.
- **Add multi-chunk questions on purpose,** so single-shot retrieval cannot pass by accident.
- **Read a sample yourself and throw out the bad ones.** Twenty minutes of this is worth more than another thousand generated questions.
- **Replace them with real ones as soon as you have real ones.** Log user questions from day one. Fifty real questions beat five hundred synthetic ones.

Synthetic questions are **scaffolding**, not a benchmark. They tell you whether you broke something between Tuesday and Wednesday. They do not tell you how good your system is.

Beyond the naive pipeline

Everything so far assumed one question, one search, one answer. Drop that assumption.

The question is not always the query

Users do not type search queries. They type this:

> **User:** What pressure does the Lori press run at?
> **System:** 2.5 bar, raised to 3.2 bar after twenty minutes [1].
> **User:** *And the other one?*

Embedding "And the other one?" retrieves nothing useful, because on its own it means nothing.

Query rewriting: before retrieving, use the conversation so far to write a standalone query - "What pressure does the PRS-380 press run at?" - and search with that. Cheap, mechanical, and the single highest-value addition to a chat-shaped RAG system.

Search with the answer you do not have

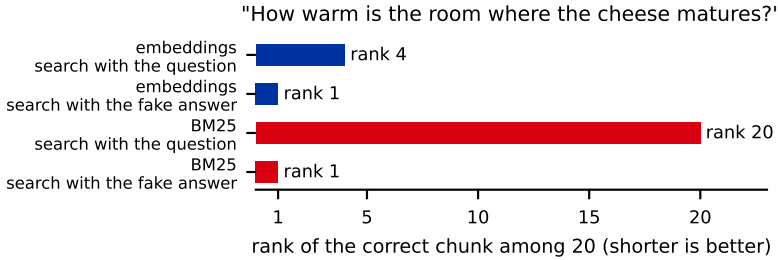
A question and its answer often share no vocabulary - the whole problem from lecture one. So do not search with the question.

1. Ask a model to *invent* a plausible answer. It will be wrong. That is fine.
2. Search with that fake answer instead of the question.
3. A fake answer looks like a passage, so it lands near real passages.

> **Question:** "How warm is the room where the cheese matures?"
>
> **Invented answer:** "The maturing room is kept cool and damp. Temperature is held at around **11 degrees** with high relative humidity so the rind does not dry out."

The real chunk says **10** degrees. The invented answer is factually wrong. **Hypothetical Document Embeddings (HyDE)** does not care - it only needs the fake answer to be shaped like the real one.

HyDE, measured on our corpus



Real measurement. Embeddings: rank 4 → rank 1. BM25 is the striking one: the question shares no content word with the chunk, so it scores **exactly 0.000** and lands last. The invented answer scores **3.857** and ranks first.

The price is an extra model call before every search: more latency, more cost, and a hallucinated passage in your pipeline that must never reach the user.

Some questions need two searches

> "What pressure does the press that replaced the PRS-220 run at?"

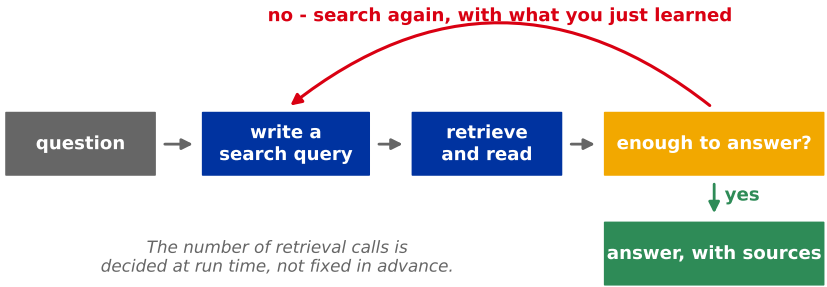
Three presses here: **PRS-220** (recalled, 2019), **PRS-400** (its replacement), **PRS-380** (harder cheeses).

No single chunk answers this. You need **"PRS-400 replaces the older PRS-220"**, and then, knowing the model number, the chunk about the press pressure. One search cannot use a fact it has not read yet.

On our 20-chunk corpus, one embedding search put the two needed chunks at ranks **1** and **8** - so a top-5 prompt is missing half the answer.

Honest caveat: BM25 happened to get both into its top 5 here, because the question contains the word "pressure". With 20 chunks that is luck, not evidence. The structural point stands: the second hop depends on a fact the first hop supplies.

Letting the system decide how many searches



Agentic retrieval: the model writes its own search query, reads the result, and decides whether it needs another one. Multi-hop stops being a special case.

You trade a fixed cost for an unbounded one. Cap the number of loops, log every query it writes, and expect a question that never terminates.

When no chunk can hold the answer

> "Which suppliers are involved in the incidents that led to a recall?"

The answer is spread across forty documents and exists only as a **pattern between** them. Top-k retrieval returns five paragraphs, and five paragraphs cannot contain a pattern over forty.

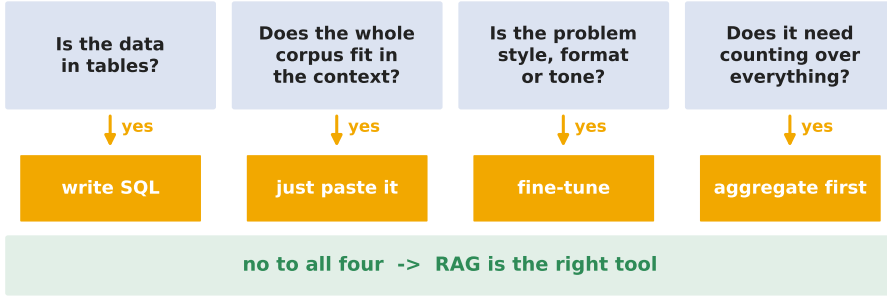
Graph RAG: while indexing, extract entities and relationships from each chunk - supplier, press, incident, recall - and build a graph. At query time, traverse the graph, then retrieve the text behind the relevant nodes.

The indexing cost is large: a model call per chunk, redone whenever the corpus changes. Use it when your questions are genuinely about connections.

When not to use RAG

The most useful thing this chapter can teach you is when to close the tab.

Four questions to ask first



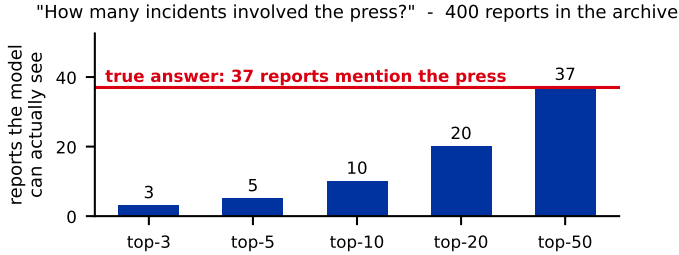
Retrieval is for **unstructured text that does not fit**. If your data is structured, small, or the problem is not about knowledge at all, you are building machinery you do not need.

The four alternatives, concretely

- **It is a database question.** "How many wheels did line two produce in March?" is a `SELECT`. Embedding your sales table is a way of getting a slower, less reliable `SUM`.
- **The whole corpus fits.** Ten policy documents, 40,000 tokens, a handful of questions a day: paste them and skip the pipeline. Retrieval is a response to *scale*.
- **The problem is behaviour, not knowledge.** If the answers are factually right but the tone, format or language is wrong, retrieval cannot fix it. That is fine-tuning, or a better prompt.
- **The question is about everything.** Counting, averaging, ranking across the whole corpus. Next slide.

Reach for RAG when the corpus is large, textual, changing, and the questions are answerable from a **few** passages. That is a narrow target, and it is a very common one.

Why RAG cannot count



Illustrative archive: 400 incident reports, of which 37 mention the press. A top-k retriever hands the model at most k of them, so the model counts k and reports it with total confidence.

The failure is **structural**, not a tuning problem. A better embedder, a reranker or a bigger model changes nothing. Count in a database, then hand the number to the model.

Recap

- **Assembly is plumbing that matters**: token budget, order, dedup on metadata.
- **The prompt is a contract**: restrict the source, cite, license a refusal. A citation is a pointer, not a proof.
- **recall@k**, at the k you really use, is the metric that matters. MRR and nDCG describe the ordering, not the outcome.
- **Faithfulness, answer relevance, context precision** need no reference answer but need a judge model, and **the judge is suspect**: 0.95 agreement with humans when purpose-built, 0.40 for "which is better?".
- **Synthetic eval sets flatter you**: recall@1 of 1.00 on copied questions, 0.38 on the same facts reworded on purpose.
- **Sometimes the answer is not RAG**: SQL, a longer prompt, or a fine-tune.

Trust **recall@k** for retrieval, **faithfulness** for the context, **your own reading** for the corpus.